Learning to Play Blackjack with Derivative Free Algorithms

1st Adhikar Chhibber
achhi002

2nd Katelyn Poon
kpoon015

3rd Hugo Wan
twan012

***Abstract*—This report aims to showcase the performance of three derivative free algorithms in the blackjack environment. In no way do we recommend using any model in any real match.**

I. Introduction

One of the most common card games in the world is Blackjack, this game is also extremely popular in reinforcement learning as many have tried to create models to "win blackjack" or as we will refer to as a "casino ready model". In our project, we took inspiration from the paper "Beating Blackjack - A Reinforcement Learning Approach" which is a paper from Stanford where they used two different methods in representing states to feed into value based methods. In this paper, their baseline performance was estimated to be a win rate of 28 percent and their best method resulted in a maximum win rate of 42.6 percent.

Our goal in this project was to experiment with derivative free methods to see if we can obtain a higher win rate than the paper's baseline model and their best model. To do this, we implemented the following algorithms: Hill Climbing, Cross Entropy Methods (also known as CEM), and Evolution Strategies.

A. Environment Details.

In all our environments, we are using gym's own blackjack environment "Blackjack-v1" with the environment giving a reward in two states: when the model wins a game (this is given a +1 reward), when the model loses (this is given a -1 reward). No reward is given if the model is given a natural blackjack (a state where the model starts with a hand that is already equal to value 21). This is a setting we enabled since the model should not be rewarded for having a winning hand.

II. Methodology

Please note in all implementations, we initialized our policy network with Xavier initialization and performed evaluation and training on random seeds to gain a more holistic policy.

A. Hill Climbing

To implement Hill Climbing, we followed the algorithm where we first create an initial policy network. We then created a perturbed network (a perturbed network is made by taking the initial policy network and adding some gaussian noise) and then collected a rollout with both networks. If the perturbed network did better than the old network, we then update the old network to the new perturbed network and continue looping until we reach a certain amount of iterations (in theory we can let this algorithm run forever and have a new network every time however due to the difficulty to have derivative free algorithms reach a certain score, it is better to let it run for a certain amount of epochs instead.

For our implementation, we decided to run the algorithm for 1000 iterations and to speed up training, we made 10 perturbed networks each iteration and then chose the best network for the final comparison.

B. Cross Entropy Methods (CEM)

To implement Cross Entropy Methods, we followed the algorithm where we created an initial policy network. We then create multiple perturbed networks and have them each collect a rollout. We then calculate the returns and choose the top k percent of networks based on their returns. Then with this top k percent, we average them all out and replace our initial policy network. We also do this up to a certain number of iterations.

For our implementation, we ran the algorithm for 500 iterations and had each iteration create 1000 perturbed networks and we took only the top 20 percent to average them into a new network.

C. Evolution Strategies

To implement Evolution Strategies, we followed the algorithm where we created an initial policy network and then created multiple perturbed networks. Each perturbed network is created by adding gaussian noise multiplied by a factor of sigma to the initial policy network. Then each network collects returns from a rollout and then the initial policy network is updated by multiplying each network by its returns and averaging them all out and then multiplying by some learning rate.

For our implementation, we decided to use the values suggested by OpenAI's own Evolution Strategies paper where they use a sigma of 0.1 and a learning rate of 0.01. The number of perturbed networks created was 1000 and the model was run for 100 epochs. Additionally in this algorithm, we add some changes to the rewards in the environment where we now give the model some reward if it chooses the action that has a high probability of winning the game. This does mean we give it an approximation of what cards are possible to be

dealt and their relative probabilities. When the model does not choose the best action, we give it a reward of -1, when it does choose the best action, we give it a reward equal to the probability of the action winning. This was done as in initial tests, the model was becoming an extremely safe player and would not have a high win rate due to this strategy, the hope of changing the reward function was to incentivize the model to take some risks.

III. Results

A. Hill Climbing Results

In the following graph, the orange line represents the mean win rate of all candidates sampled at each step, which stabilizes around 0.38. The blue line tracks the best win rate achieved so far, eventually reaching a maximum of 0.466 during training. After training, a 20,000-episode evaluation confirmed that the final policy achieves a stable win rate of 0.382, indicating that the peak values during training represent exploratory noise rather than long-term performance.

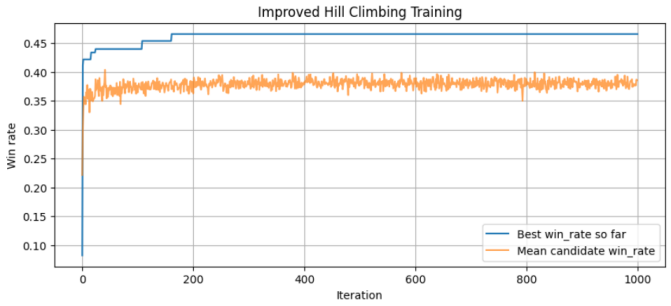


Fig. 1. Hill Climbing Training Curve.

In the following figure, the random policy achieves a baseline win rate of 28.2 percent, while the improved Hill Climbing model reaches 38.3 percent over 20,000 games. Multi-seed evaluation produces a nearly identical win rate (38.3 percent), confirming the stability and reproducibility of the final policy.

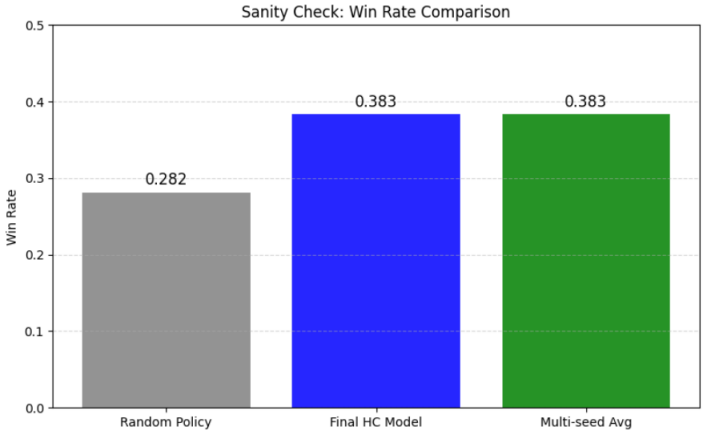


Fig. 2. Win Rate Sanity Check.

Example frame from the Hill Climbing policy visualization. The dealer shows a 10 while the player holds a total of 13, illustrating one of the decision points captured in the animated rollout.

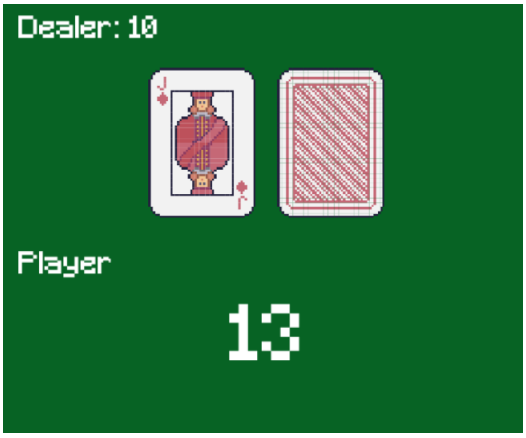


Fig. 3. Example Game with Hill Climbing.

B. Cross Entropy Methods (CEM)

As shown in figure 4. CEM generally achieved its highest performance within the first few epochs (39.9 percent win rate) before suffering a large drop. Afterwards, it had a general increasing trend towards 38.4 percent at the later epochs. This suggests that the model gets an initial lucky start at the beginning before deviating from that initial policy. The performance shows a large amount of variance which is expected of the CEM algorithm. The average training reward in figure 5 also shows the same initial pattern of peaking at -0.42 before quickly spiking down within the first few epochs. Unlike the average evaluation, the average training reward appears to stabilize more around -0.52. As for the bust rate in figure 6, the CEM model shows an interesting pattern of staying at 0 for the entire run. The CEM model apparently chooses a policy that avoids busting in an attempt to maximize rewards by playing it safe. This could explain why the performance generally stays stuck in the 38 percent range with very little improvement.



Fig. 4. CEM Average Training Curve.

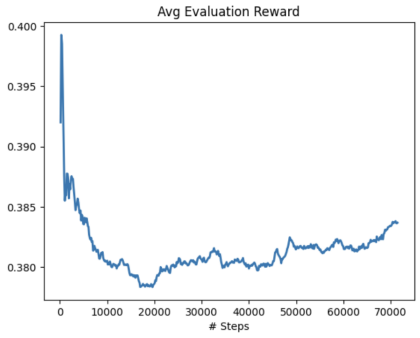


Fig. 5. CEM Average Evaluation Curve.

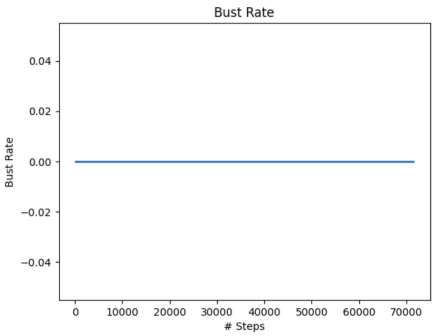


Fig. 6. CEM Bust Rate Curve.

C. Evolution Strategies

Initially Evolution Strategies was built without any changes to the reward function and this algorithm had achieved a maximum of 39.2 percent win rate. This was then further examined and it was shown that the model was learning to never go over 21 and would learn to take the safest path in blackjack. This leads to the model not taking risks and just losing to the dealer each time. So some modification to the reward function was made and the model was reward tuned more carefully to ensure that good actions were rewarded instead of just good games. That way even a loss can still show the model how to act properly. The model then achieved a maximum win rate of 43.2 percent. Making this model higher than the models produced by the inspiration paper. To prove this model was successful, we ran 20 million evaluation rollouts on it to confirm its win rate.



Fig. 7. Training Curve of Evolution Strategies.

As we can see in figure 7, the model has an extremely unstable training curve however it is going in the positive direction, suggesting that the model is making better actions as it trains.

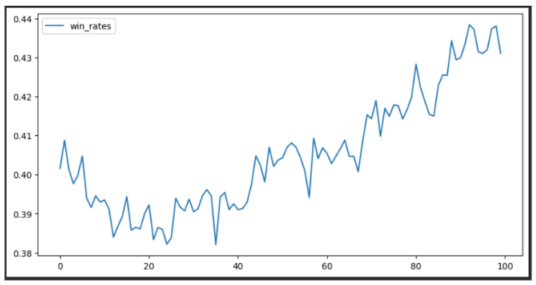


Fig. 8. Evaluation Curve of Evolution Strategies.

As we can see in figure 8, the model starts with a win rate initially above 40 percent and this is disregarded as we determine that the model has not really learned how to play a game properly.

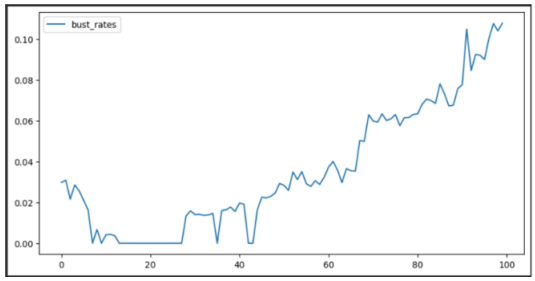


Fig. 9. Bust Rate of Evolution Strategies.

In figure 9, we see the model's bust rate which is the rate at which the model goes bust in games that it loses. Here we see actually some connections to our win rate. When the model reaches 0 in bust rate, the model typically is below 40 percent win rate. When it is above, the win rate increases. Of course this does not mean a higher bust rate means a higher win rate, however it does mean that having a slightly higher bust rate than 0.04 suggests that the model will perform better. In our case we describe it as the model learning to take risks and as we can see from the graphs above, taking some level of risk does lead to more rewardable rollouts and a higher win rate in evaluation rollouts.

In figure 10 we see an example game the model plays with a rather tricky situation given to it. Since the dealer is showing 10, it is safe to assume that the dealer has 15 or above. In this situation, the model also has 17 which makes this a specific situation we wanted the model to see: what to do if the probability of loss is great when taking a hit and staying. In this case the model chooses to stay which is a more safe yet sound decision to make. This model does lose the game however it takes the best possible action.

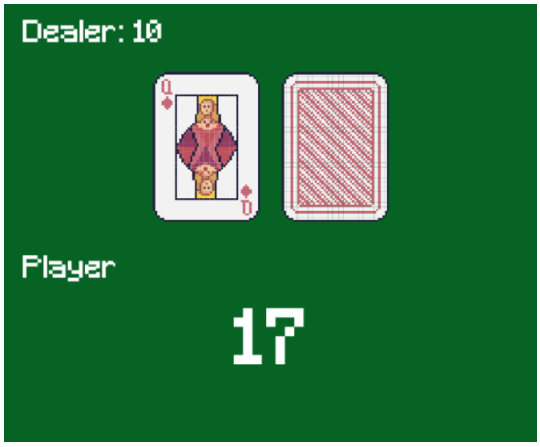


Fig. 10. Sample game with Evolution Strategies.

IV. Code Status

All models are currently considered to be complete with us considering one model to have ”solved” blackjack. All code associated with these models can be run without error and with/without GPU.

V. Conclusion

Our Hill Climbing approach successfully learned a blackjack policy that consistently outperforms the random baseline. While the model's peak training win rate briefly reached 0.466, its stable performance converged to approximately 38.3 percent across large-scale and multi-seed evaluations, confirming both reliability and reproducibility. Although Hill Climbing lacks gradient information and therefore improves more slowly than modern RL algorithms, it still demonstrated meaningful learning through iterative perturbations and provided a solid, interpretable baseline for comparison with more advanced methods.

Our CEM approach was also able to successfully learn a blackjack policy that consistently outperforms the baseline. While the model's best mean eval reward was 39.9 percent before quickly dropping, its performance was able to steadily rise back to 38.36 percent within 400 epochs. CEM was generally finding a better policy at the beginning, but it does show that it can improve on its policy with time and random seeds after deviating from the original policy.

Our Evolution Strategies model not only was the model that had the highest win rate, this model outperformed our inspiration's models by 0.8 percent. With our evolution strategies model having a peak performance of 43.2 percent win rate (this was further tested with 20 million evaluation rollouts). We can conclusively say this model has learned how to play blackjack in a less conservative way than the other models, however it can match the human performance of 42 percent. Proving that derivative free algorithms can perform well in this environment where the expectation is that the player will lose.

VI. Future Directions

Our future current direction is to explore what other kinds of models can be used within this environment and to see if other derivative free algorithms can achieve similar or higher performance. We also want to test these models on different blackjack environments where other rules are used during play such as ”double down” and ”split”. Along with a budget counter to actively track the potential gain/loss from these models.

References

[1] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution Strategies as a Scalable Alternative to Reinforcement Learning,” arXiv:1703.03864 [stat.ML], 2017.

[2] J. Geiser and T. Hasseler, “Beating Blackjack — A Reinforcement Learning Approach,” Stanford University, 2020. Available: https://web.stanford.edu/class/aa228/reports/2020/final117.pdf

[3] A. Khare, “Cross-entropy method for Reinforcement Learning,” Medium, Feb 2020 Available: https://medium.com/data-science/cross-entropy-method-for-reinforcement-learning-2b6de2a4f3a0

[4] Thomas, M., O'Boyle, N.M., Bender, A. et al. Augmented Hill-Climb increases reinforcement learning efficiency for language-based de novo molecule generation. J Cheminform 14, 68 (2022). https://doi.org/10.1186/s13321-022-00646-z

[5] S. Pappala, “Hill-Climbing Search Artificial Intelligence,” *International Journal of Scientific Research in Engineering and Management (IJSREM)*, vol. 7, no. 7, pp. 1–?, Jul. 2023. [Online]. Available: https://www.researchgate.net/publication/378679524_Hill-Climbing_Search_Artificial_Intelligence